Kick Start 2022 - Coding Practice with Kick Start Session #2

Analysis: Building Palindromes

View problem and solution walkthrough video

For each of Bob's questions, we have to figure out if it is possible to rearrange a string of letters to form a palindrome. The set of letters is the substring $[\mathbf{L_i}, \mathbf{R_i}]$ from the original string from the test case.

One possible approach is to enumerate all permutations of the substring and check if any are palindromes. For a substring of length l, this would involve checking up to $Factorial(l)$ permutations, each of which costs $O(l)$ to check if the permutation is a palindrome. Sometimes, "brute force" solutions like this are sufficient to solve competition problems, but in this case, even the $\mathbf{N}$-max of 20 from the smaller test set would take on the order of $20 \times Factorial(20) \approx 4.9 \times 10^{19}$ operations to perform. This is orders of magnitude more than can be accomplished within the time limit.

One important observation is that a palindrome can be described as an arbitrary string of zero or more letters, followed by the reverse of that string, optionally with one extra letter between them. For example, `ABCCBA` (without a middle letter) and `ABCXCBA` (with a middle letter) are both palindromes. These examples illustrate an important property of palindromes, which is that all letters, with the exception of the optional middle letter, must occur an even number of times.

This property holds true for all palindromes, and it turns out that a set of letters with this property can *always* be arranged to form a palindrome. We can do this by selecting pairs of letters and using them to construct two identical strings. Eventually, we will have at most one letter left over. Finally, we can take one of the strings and concatenate the reverse of the other, optionally with the leftover letter in between. The resulting string will be a palindrome!

Test set 1

For each question, we can count the frequencies of all letters in the substring and decide if it is possible to rearrange them to form a palindrome. For each question, the cost of calculating the frequency of each letter is $O(\mathbf{N})$. With $\mathbf{Q}$ questions per test case, this approach has a complexity of $O(\mathbf{Q} \times \mathbf{N})$, which is sufficient for Test set 1.

Test set 2

This test set has much larger limits for $\mathbf{Q}$ and $\mathbf{N}$, so the approach from Test set 1 will be too slow to complete within the time limit. An important observation is that for a given test case, Bob's questions are not independent - rather, they are all questions about substrings of the *same* string. Can we use this to our advantage and share or reuse calculations among the questions?

Thinking about the approach used in Test set 1, we can observe that the process of counting the letters in each substring ends up being repeated many times between the different questions. For example, if one question asked about the range $[20, 70]$ and another asked about the range $[30, 80]$, these both involve repeating the same counting process for the range $[30, 70]$. It turns out we can perform the counting process for *every possible question* ahead of time by generating a prefix sum for the frequency of each letter.

Generating a prefix sum for the letter frequencies in a string of length $\mathbf{N}$ has a complexity of $O(\mathbf{N})$. Furthermore, once the prefix sum is generated, answering any of Bob's questions can be done in constant time! For any question $[\mathbf{L_i}, \mathbf{R_i}]$, the frequency of letters in the substring is just the difference in value between the prefix sum at the beginning and the end of the substring. This reduces the computational complexity of each test case from $O(\mathbf{Q} \times \mathbf{N})$ to $O(\mathbf{Q} + \mathbf{N})$, which is sufficient for Test set 2.

It is also worth mentioning that while the 1GB memory limit is more than sufficient for this approach, we don't actually need to store the letter frequencies themselves. Because we only need to check whether a letter occurs an even or odd number of times, we can actually calculate a "prefix parity" and compare the parity at the beginning and end of a substring. If at most one letter has a different parity, then the substring can be arranged to form a palindrome.

Example Solution

```
Solve(N, Q, letters, questions) {
  answer = 0
  prefix[1]['A':'Z'] = 0
  for i in 1:N {
    prefix[i+1] = prefix[i]
    prefix[i+1][letters[i]]++
  }
  for i in 1:Q {
    [L,R] = questions[i]
    frequencies = prefix[R+1] - prefix[L]
    odds = 0
    for c in 'A':'Z' {
      if IsOdd(frequencies[c]) {
        odds++
      }
    }
    if odds <= 1 {
      answer++
    }
  }
  return answer
}
```

Kick Start 2022 - Coding Practice with Kick Start Session #2

Analysis: Irregular Expressions

View problem and solution walkthrough video

The problem translates to determining whether the given string contains a substring (`spell`) satisfying the given conditions:

- The spell must have 3 "words": `start, middle, end`
- The start word must equal end word, and must be at least 2 syllables
- The middle word must have at least 1 syllable

Recall that a word is defined as a string of characters of any length with at least one syllable. A syllable is a string that must have at least one vowel in `a, e, i, o, u` and can be of any length.

Let $S = E$ be the start and end word, and M be the middle word. Let v_i represent the i-th vowel in the string.

We can make a few observations and begin iterating through the string.

1. We can assume that S begins at v_1. We can ignore all the consonants before v_1, and assume they are either not part of the spell (in the case for S), or they are part of M (in the case for E), since M's only constraint is that it contains at least 1 vowel.
2. We must assume that S stops at v_2. This is the first string we encounter which satisfies the constraint that S has 2 syllables, and stopping here ensures we do not miss any valid spells. Similar to the above observation, we can do this because we can simply tack on any extra consonants after S to M, and any extra consonants after E does not matter.
3. Then, we can start looking for M. Once we see v_3, we should stop. We have now satisfied the only constraint for M. Note that M can have more vowels or consonants after v_3, and no constraints will be broken.
4. After seeing v_3, we can begin our search for E. We already established what substring E must be, so we can simply do a search on the rest of the string starting from the character after v_3. If S appears, then our search is finished. Otherwise, a spell with $S = E$ does not exist. In this case, we need to repeat the process and look for a new S and E.

If we need to look for a new S and E, where should we start? We can look for a new S starting at v_2 and ending and v_3, and follow the same logic as above. If we iterate through all the vowels and still do not find a valid spell, then we know that the string does not contain one, and can return `Nothing`.

Now let us look at the runtime of this solution. Let $\mathbf{N}$ be the length of the given string. We iterate through the entire string to look for a valid S, M, and E, at most $\mathbf{N}$ times in the worst case. Thus, the runtime is $O(\mathbf{N}^2)$.

Another simple solution is to use RegEx to search whether a spell satisfies the given constraints. The runtime of this solution is $O(\mathbf{N})$, but keep in mind that the RegEx construction could take up to $O(2^{\mathbf{C}})$ construction time and space, where $\mathbf{C}$ is the size of the regex.

Kick Start 2022 - Coding Practice with Kick Start Session #2

Analysis: Parcels

View problem and solution walkthrough video

Test Set 1

For Test Set 1, we are able to check all possible locations for the new delivery office in order to find the one that minimizes delivery time. We will do this in two stages. First, we will compute the delivery time for each square given the existing delivery offices. Second, we will try all possible locations for the new office. The precomputation in the first part will allow us to find the delivery time in the second part more efficiently.

For the first stage, we compute the delivery time of a square by iterating over the entire grid and finding the minimum manhattan distance to a square that has a delivery office. This has a time complexity of $O(\mathbf{RC})$ per square for a total time complexity of $O((\mathbf{RC})^2)$.

For the second stage, we iterate over all possible locations for the new delivery office and for each location, search the entire grid for the square with the maximum new delivery time. The new delivery time is the minimum of the delivery time computed in the first part and the manhattan distance to the new delivery office. This also has a time complexity of $O(\mathbf{RC})$ per delivery location for a total time complexity of $O((\mathbf{RC})^2)$, which is sufficient for Test Set 1.

Alternatively, we could skip the first stage if we use a faster way of computing the delivery time for each square such as breadth-first search. See the next section for more details.

Test Set 2

We can use a similar approach for Test Set 2, however we will need a more efficient way to compute the maximum delivery time for a new delivery location. We will do this by solving the following subproblem: given a value of $\mathbf{K}$, can we add a new delivery office so that the maximum delivery time is at most $\mathbf{K}$? The solution to this subproblem will be similar to Test Set 1, however having a target value of $\mathbf{K}$ allows us to identify exactly which squares need to be serviced by the new delivery office. We can use this difference to create a faster solution.

Note that if the answer to the original problem is $\mathbf{K}$, then the answer to our subproblem will be `'No'` for values in the range [1, $\mathbf{K}$-1] and `'Yes'` for values in the range [$\mathbf{K}$, `infinity`]. For these kinds of subproblems, we can use binary search: if a given value works, then it is an upper bound for the answer; otherwise it's a strict lower bound for the answer. Hence, once we have a solution for the subproblem, we can use binary search to solve the original problem. This is a common technique to transform optimization problems into decision problems.

First, we efficiently compute the existing delivery time of every square by inverting the problem: instead of finding the shortest distance to a delivery office for each square, we find the shortest distance to each square from a delivery office. This can be done using a multiple-source, breadth-first search starting at all of the delivery offices. A multiple-source BFS is the same as a regular BFS except that you use multiple starting locations instead of one. This search visits each square at most once, which gives us a time complexity of $O(\mathbf{RC})$.

Second, we identify all of the squares which have a delivery time greater than $\mathbf{K}$ and then determine if there exists a location that is within a distance of $\mathbf{K}$ to each of these squares. In order to do this efficiently, we note that the manhattan distance has an equivalent formula:

dist(($\mathbf{x_1}$, $\mathbf{y_1}$), ($\mathbf{x_2}$, $\mathbf{y_2}$)) = max(abs($\mathbf{x_1}$ + $\mathbf{y_1}$ - ($\mathbf{x_2}$ + $\mathbf{y_2}$)), abs($\mathbf{x_1}$ - $\mathbf{y_1}$ - ($\mathbf{x_2}$ - $\mathbf{y_2}$)))

This formula is based on the fact that for any point, the set of points within a manhattan distance of $\mathbf{K}$ form a square rotated by 45 degrees. The benefit of this formula is that if we fix ($\mathbf{x_2}$, $\mathbf{y_2}$), the distance will be maximized when $\mathbf{x_1}$ + $\mathbf{y_1}$ and $\mathbf{x_1}$ - $\mathbf{y_1}$ are either maximized or minimized.

Hence, we can compute the maximum and minimum values of both $\mathbf{x_1}$ + $\mathbf{y_1}$ and $\mathbf{x_1}$ - $\mathbf{y_1}$ for all squares with a delivery time greater than $\mathbf{K}$. Then, we can try all possible locations for the new delivery office and check if the maximum distance from the location to a square with a current delivery time greater than $\mathbf{K}$ is at most $\mathbf{K}$ in constant time. Hence, we can check if the answer is at most $\mathbf{K}$ with a time complexity of $O(\mathbf{RC})$.

With the binary search, the time complexity becomes $O(\mathbf{RC}\log(\mathbf{R}+\mathbf{C}))$, which is sufficient for the test set. There is a way to improve this to $O(\mathbf{RC})$ time by computing the min/max values mentioned above for all possible $\mathbf{K}$ in a single pass over the grid and then using casework to determine if a viable new delivery office location exists for each $\mathbf{K}$, but this optimization is unnecessary.

Kick Start 2022 - Coding Practice with Kick Start Session #2

Analysis: Sample Problem

View problem and solution walkthrough video

Solving this problem involves a number of steps. First, we must determine the total number of candies we have available to distribute. We can calculate this by summing together the number of candies in each bag, $total := \sum_{i=1}^{\mathbf{N}} \mathbf{C_i}$. Once we have the total number of candies, we must determine the number of candies that will remain after evenly distributing as many as possible among $\mathbf{M}$ kids. We can calculate this by dividing the total number of candies by the number of kids. $\mathbf{M}$ may not evenly divide $total$, so $(total \div \mathbf{M})$ will have an integer part (the amount each kid receives) and a remainder part (the amount left over). Because we only care about the remainder, we can use the modulo operation which does exactly this.

Example solution:

```
Solve(N, M, C) {
  total = 0
  for i in 1:N {
    total = total + C[i]
  }
  remainder = modulo(total, M)
  return remainder
}
```

Limits and complexity:

With the input limits for this problem, the default integer size for most programming languages will be sufficient to perform all arithmetic without risk of overflow. Specifically, the largest possible value is $total = (\mathbf{N_{max}} \times \mathbf{C_{i_{max}}}) = 10^8$, which is easily stored in a 32-bit signed integer ($int_{max} \approx 2.147 \times 10^9$). If the limits were larger, we could take advantage of the transitive and symmetric properties of modular arithmetic to calculate the remainder provided by each bag, and the running remainder after each addition:

```
Solve(N, M, C) {
  remainder = 0
  for i in 1:N {
    remainder = remainder + modulo(C[i], M)
    remainder = modulo(remainder, M)
  }
  return remainder
}
```

This reduces the largest intermediate value to $2 \times (\mathbf{M_{max}} - 1)$ with only a small amount of extra computation. When solving other problems in this competition, you are likely to encounter limits that will require this kind of optimization.

Computational complexity is also an important factor to consider when deciding how to approach a problem. Both approaches presented here have the same complexity, $O(\mathbf{N})$, which is sufficient to solve the problem within the time limit. Other problems in this competition can often be solved using a variety of approaches, but only some will be sufficiently fast to solve all test sets within the time limit. An algorithm with complexity $O(\mathbf{G}^3)$ may work for a small test set with $\mathbf{G}_{max} = 20$, but that same algorithm will be too slow for a large test set with $\mathbf{G}_{max} = 10^6$.

In general, time and memory limits for the problems in this competition, and the bounds for their test sets, are chosen so that algorithms pass or fail based on their overall time and memory complexity order. In other words, if a problem has a time limit of 60 seconds, it is unlikely that a fast algorithm would take 40 seconds while a slow one took 80 seconds. Instead, you will find that a fast algorithm might take 5 seconds while a slow one takes 5 years. So if you find your solution exceeding the time limit for a problem, take a moment to reevaluate your algorithm. There may be another approach that is many times faster.

Kick Start 2022 - Coding Practice with Kick Start Session #2

Analysis: Sherlock and Parentheses

View problem and solution walkthrough video

The problem translates to finding the maximum number of balanced non-empty substrings possible in a string, generated from the given number of left and right parentheses. A string S consisting only of characters `(` and/or `)` is *balanced* if:

- It is an empty string, or:
- It has the form (S), where S is a balanced string, or:
- It has the form S_1S_2, where S_1 is a balanced string and S_2 is a balanced string.

Test Set 1

Naively, we can generate all the permutations of strings possible from $\mathbf{L}$ left parentheses `(` and $\mathbf{R}$ right parentheses `)`. The time complexity of generating all the permutations will be in order of possible permutations i.e. $O(\frac{(\mathbf{L}+\mathbf{R})!}{\mathbf{L}!\times\mathbf{R}!})$.

For counting the number of balanced non-empty substrings in a string, we can use a basic counter which increases by 1 for a left parenthesis `(` and decreases by 1 for a right parenthesis `)`. While iterating a given string, if the value of the counter becomes negative before reaching the end, then this string cannot be a balanced string because it denotes the occurrence of right `)` parenthesis before it's matching left `(` paranthesis. If the counter is 0 at the end of the string, avoiding being negative throughout the string then it denotes a balanced string. We can do this check for all the substrings in a efficient way using a nested for loop, which results in a Time complexity of $O(N^2)$ where N is the length of the string:

```
int CountBalancedSubstrings(String S) {
  int N = S.length();
  int balanced_substrings = 0;
  for (int i = 0; i < N; i++) {
    int counter = 0;
    for (int j = i; j < N; j++) {
      if (S[j] == '(')
        counter++;
      else
        counter--;
      if (counter < 0) break;
      if (counter == 0) balanced_substrings++;
    }
  }
  return balanced_substrings;
}
```

So overall, we can generate all the permutations of the string, count the balanced substrings and return the maximum possible number of balanced non-empty substrings in $O(\frac{(\mathbf{L}+\mathbf{R})!}{\mathbf{L}!\times\mathbf{R}!} \times (\mathbf{L}+\mathbf{R})^2)$ Time complexity.

Test Set 2

We cannot generate all the possible permutations of strings as the solution would timeout. We cannot even count the number of balanced substrings in a string using our earlier $O(N^2)$ algorithm as N can go upto 10^5.

We need to identify the optimal strategy to get the maximum number of balanced substrings. We can observe few things from our CountBalancedSubstrings algorithm:

- The counter should not become *negative* while iterating throughout the string.
- The number of balanced substrings we get is directly proportional to the number of times the counter becomes 0.

So this translates to the optimal strategy of arranging left parentheses `(` and right parentheses `)` like this `()()()()()`.....
Now every left parenthesis requires a right parenthesis and vice versa for forming a balanced string. So we can say this pattern will follow till $2 \times N$ length where $N = min(\mathbf{L}, \mathbf{R})$, and the rest of the string would not be a part of any balanced substring as it would either include all left `(` parantheses or all right `)` parantheses.
Now in this pattern we can see that the first left parenthesis `(` forms a balanced substring with all the N right parentheses `)` ahead of it. Similarly, every left parenthesis `(` forms a balanced substring with all the right parentheses `)` ahead of it till $2 \times N$ length. So the total number of balanced non-empty substrings will be $1 + 2 + 3 + 4 + \cdots + N = \frac{N \times (N+1)}{2}$

So for each test case, we can calculate N and then the maximum possible number of balanced non-empty substrings in $O(1)$ Time and Space complexity.